Litterbox PDM

User Guide

Check-in, check-out, revisions, releases, and BOMs for small SolidWorks teams, on Git and Git LFS.

For Litterbox 1.0.0 · October 2026

Contents

What Litterbox is 4
- Terms 4

Getting started 6
- What you need 6
- Installing Litterbox 6
- Joining your team's vault 7
- Decisions to make first 7
- Setting up a new team 8
- The main window 8
- Your first part 9

Everyday work 10
- Part numbers and file names 10
- Projects 10
- Finding parts 11
- Folders 11
- Checking out and checking in 12
- Getting other people's changes 12
- Opening files 12
- The details pane 13

Revisions and the lifecycle 14
- Form, Fit, Function, and Dispo 14

Assemblies, references, and BOMs 15
- Uses and where used 15
- The BOM tab 15
- Toolbox parts 15
- Non-CAD parts and kits 16

Purchasing, costs, and materials 17
- Purchasing info 17
- Materials 17

Release outputs and exports 18
- STEP and PDF files 18
- Exporting 18

Documents 19

Sign-off workflows and ECOs 20
How workflows work 20
Asking for sign-off 20
Reviewing a request 20
Rejected requests 21
Engineering change orders 21
Running the team 22
People and access 22
Vault settings 22
The team's Toolbox 23
Vault maintenance 23
Importing existing CAD 24
Working with SolidWorks 25
Configurations 25
Templates 25
How Litterbox works 27
The vault is a Git repository 27
Locks, read-only files, and uploads 27
Downloads 28
The job queue 28
Settings and troubleshooting 29
Your settings 29
Problems 29
The command line 30
Quick reference 31

What Litterbox is

Litterbox is a product data manager (PDM) for small hardware teams that design in SolidWorks. It keeps the team's CAD files in one shared vault, lets only one person edit a part at a time, numbers parts, tracks revisions and releases, and works out bills of materials.

Litterbox has **no server**. A vault is an ordinary Git repository that uses Git LFS (Git's add-on for large files), hosted on a Git host such as GitHub, Azure DevOps, or GitLab. Litterbox is a Windows program that turns Git's commits and file locks into check out, check in, release, and revise. You don't need to use Git yourself.

This design has the following consequences:

- **Cost.** The only cost is the Git host, which is often free.
- **Offline work.** Downloaded files are on your own disk. You need a connection only to check out, check in, and get other people's changes.
- **History.** Every change is a commit that records who, when, and why. The host keeps a copy.
- **Readable data.** Part records are plain text (YAML) files. They can be read and scripted without Litterbox.

Litterbox is meant for groups of a few to a few dozen people, such as student competition teams, small companies, labs, and makerspaces, that need release control and a lasting history without running a PDM server.

Terms

Term	Meaning
Vault	The shared Git repository that holds the team's parts, and its copy on your computer under C:\Litterbox\<vault name>.
Org	The team or company a vault belongs to. Its short code (HCR) starts every part number. You can belong to several orgs and switch between them.
Part number	A part's permanent identity, for example HCR-100010-001: the org code, a base number, and a variant. A number is never reused, even after the part is voided.
Revision	A released or to-be-released version of a part: A, B, C, and so on. A part's file names include it (HCR-100010-001_A.SLDPRT).
Record	The text file that describes a part: its name, status, revisions, history, purchasing info, and the parts it uses. Records are in the PN-logs folder.
Check out / check in	Taking the lock on a part so you can edit it, and uploading your changes and releasing the lock.
Release	Freezing a revision. Released files can't be changed. To change the part, revise it, which starts the next revision.
Workflow	A set of sign-offs, by role, that an action needs before it happens.
Project	A block of part numbers set aside for one project. A project can have a Lead and members, and can be restricted or hidden.
Folder	A named list of parts, documents, and ECOs in Home. It only points at them, so a part can be in any number of folders.

Term	Meaning
Template	A SolidWorks file that new parts, assemblies, or drawings start from (.prtdot, .asmdot, .drwdot). The org has its own, a project can have more, and one of each kind is the default.

Getting started

If someone has already set up your team, read What you need, Installing Litterbox, Joining your team's vault, and Decisions to make first, then go to The main window. If you are setting up a team, read the whole chapter.

What you need

- A Windows computer.
- **Git for Windows** (git-scm.com), which includes Git LFS. Install it with the default options.
- An account on your team's Git host with access to the vault's repository. Your team lead adds you.
- **SolidWorks**, to model. Without it you can look at parts, open PDFs, run workflows, and manage purchasing and documents. Anything that reads or writes SolidWorks files needs it.

Git puts your name and email on every change, and can't save changes without them. When a vault opens and git doesn't know them, Litterbox asks, filled in from the team list, and sets them for that vault only. If git has a different name from the team list's, Litterbox offers to use the team list's. You can also set them for every repository in a terminal:

```
git config --global user.name "Alex Smith"
git config --global user.email "alex@example.com"
```

Installing Litterbox

1. Download Litterbox-Setup-<version>.exe from the latest release on the Litterbox GitHub page.
2. Run it. It installs for your Windows user only and doesn't need administrator rights.
3. Windows SmartScreen may say the app is unrecognised, because the installer isn't code-signed. Choose **More info**, then **Run anyway**.

Litterbox checks for a newer version at start-up and asks before it updates. Updates are signed, and Litterbox refuses an unsigned one. To turn off the check, add `update_check: false` to C:\Litterbox\settings.yaml. Help > Check for updates... checks by hand.

- **Betas.** Help > **Get beta versions** offers betas as well as releases: the next version before it is finished, for trying new things early. Expect more bugs, and report them with Help > Report a problem. A beta can make a vault need the version it leads to, which teammates on the released version don't have yet, so try betas in a test vault, or move the whole team at once. When that version is released, you are offered it like everyone else. Turning betas off keeps the beta you have until a newer release comes out.
- **Signed notices.** With each check, Litterbox also reads a notice signed by the Litterbox maintainers. It can withdraw an update key (if one were ever stolen, Litterbox stops trusting it straight away) or say that Litterbox's releases have moved; Litterbox remembers either in settings.yaml and says so.
- **Licences.** Litterbox is free software under the MIT licence. The licences of the other programs it is built with are in the licenses folder beside Litterbox.exe (in %LOCALAPPDATA%\Programs\Litterbox).

Joining your team's vault

1. Get the vault's repository URL from your team lead: the https address from the host's Clone button.
2. In Litterbox, choose **File > Add org...** and paste the URL. Litterbox copies the vault into C:\Litterbox\<repository name> in the background and opens it.
3. Your team lead adds you to the team list (Team screen > Add Person) with your host user name. Until then, you can look at everything but can't change anything.

Litterbox **downloads on demand**. A new copy gets every part record and its history, and leaves the CAD files on the host until they are needed. Checking out a part downloads its files. Opening a file downloads what it needs first (for an assembly, every part it uses). Vault > Download All... gets everything, and `download: all` in settings.yaml keeps it that way.

If your team uses **hidden projects**, the parts of a hidden project you are not a member of are left out of your copy (see Projects).

> Don't move or rename the vault folder. Every computer keeps vaults under C:\Litterbox, so SolidWorks assemblies find their parts on every machine.

Decisions to make first

Some choices are built into a team's CAD files and history as soon as parts are released, and a later update of Litterbox can't undo them. Decide these before the first part:

Decision	Recommendation
Toolbox fasteners	Don't use SolidWorks Toolbox. Model each fastener as a part of its own, so it has a part number, revisions, and purchasing info like any other part. A new vault refuses Toolbox parts (the Toolbox rule, in Vault settings). If you must use Toolbox, an Admin keeps the team's own copy in the vault (see The team's Toolbox), and everyone points SolidWorks at it.
Configurations and design tables	Don't keep several configurations in one file: model each one as a separate part. Litterbox gives each file one part number, one material, and one BOM line, and a file can't be split into several part numbers later. If linked tables (Excel design tables or equation files) define a part's geometry, keep them as controlled documents, so a change to them is released like any other. Revising such a document gives its file a new name (_B), so point the parts at the new file when they are next revised.
Part number digits	Use at least 6 digits in the base number, and more for a larger team. The digits can't be changed after the vault is set up.
One vault	Keep one vault for the whole team and all its projects. An assembly can only use parts in its own vault, and parts can't move between vaults. Use projects to separate work inside it (see Projects for what Restrict and Hide do and don't protect).
Git host	For a team of more than three or four members, consider GitLab Premium. Free plans limit the number of members or the Git LFS storage. Moving to another host later is possible but takes planning (every part must be checked in first), and files can't be removed from the vault's history, so its storage only grows.

Setting up a new team

One person sets up the team and becomes the vault's **Owner**. The org code and the part number format can't be changed afterwards, so decide them first (see Decisions to make first).

1. On your Git host, create an **empty** repository (no README) for the vault. The host must support Git LFS file locking; GitHub, Azure DevOps, and GitLab do. Hosts limit LFS storage differently: GitHub's free allowance is small, and Azure DevOps has no practical limit.
2. In Litterbox, choose **File > Create new org...** and paste the repository's URL.
3. Fill in the org's **code** (2 to 4 letters), its full **name**, the number of **digits** in the base and variant numbers (6 and 3 give HCR-100010-001), the **first number** to hand out, and the variant where assemblies start (500 by default).
4. Check the summary. Litterbox then sets up the repository (the Git LFS and lock rules, the settings file, and the team list with you as Owner) and uploads it as the vault's first commit.

Then set up the team on the **Team** screen:

- **Add Person** for each member: name, email, user name(s) on the Git host, access level, and roles.
- **Roles...** for the jobs people do (Responsible Engineer, Manufacturing, Quality, Manager, Chief, and so on). Workflows ask for sign-offs by role. **Add the standard roles** adds a common set.
- **Part Workflows...** and **Doc Workflows...** if actions need sign-offs (see Sign-off workflows and ECOs).
- **Projects...**, **Document Types...**, **Toolbox...**, and **Vault Settings...** as needed.
- **Save** at the top right. Changes to people, roles, and workflows are drafted on the Team screen and uploaded together.

To bring in existing CAD files, use Create > **Import...** (see Importing existing CAD).

The main window

Area	Purpose
Menu bar	File (orgs, exit), Vault (refresh, get latest, downloads, file rules, vault size, new parts), View (screens, sidebar, details pane, log, columns, text size), and Help. The org's name is in the middle.
Header	The org box (switch orgs), who you are, and how many changes others have uploaded that you don't have yet. Click the count to Get Latest.
Sidebar	Views of the parts list (All, Mine, Checked out, Released, Not downloaded, and Void, with counts), then the Workflows, ECOs, Documents, and Team screens. Ctrl+1 to Ctrl+5 switch between them.
Toolbar	Check Out, Check In, Save Progress, and Undo Checkout, then the Part, Lifecycle, and Create menus. Hover over a greyed-out button to see why it is unavailable.
Home / Vault	The switch left of the search box. Vault shows the parts list; Home shows folders (see Folders).
Parts list	One row per part: a tick box and status icon, then Part number, Rev, Name, Description, and more. Each status has its own icon shape (Help > Status icons).
Details pane	Everything about the highlighted part, in tabs. The button at its top, or double-clicking a part, opens it in a separate window.

Area	Purpose
Status bar	A dot showing ready, working, or error, the last result, the queued jobs, and the Log.

Ticking a part and **highlighting** it are different. Actions that change parts (Check Out, Release, and so on) act on ticked parts. Actions that only look (Open, References, Details) act on the highlighted row. Right-clicking a part ticks only that part and opens its menu.

Actions run in the background. You can keep browsing, ticking, and queueing actions during an upload. A part with a queued job shows it in its status.

Your first part

1. Choose Create > **New Part**. Enter a name and, optionally, a description and project. Litterbox suggests the next free part number; you can type another.
2. Choose its **Template**: the default is chosen for you (see Templates). Litterbox creates the part's record, checks it out to you, makes its model from the template under the right name (for example HCR-100010-001_A.SLDPRT), and opens it in SolidWorks.
3. Model the part and save it. For a drawing, choose Create > **New Drawing...**: Litterbox makes HCR-100010-001_A.SLDDRW from the drawing template, as an empty sheet, and opens it.
4. In Litterbox, tick the part and press **Check In**, with an optional comment. Litterbox uploads the files and the record and releases the lock.

Other people see the part after their next Get Latest.

With **None** as the template (or without SolidWorks on your computer), Litterbox instead shows the file name the model must have, with a Copy button: in SolidWorks, **Save As** into the vault's cad folder (C:\Litterbox\<vault>\cad\) under that name. A drawing has the same name with .SLDDRW.

Everyday work

Part numbers and file names

A part number is the org code, a base number, and a variant: HCR-100010-001. Related parts can share a base with different variants. New Part uses variants below the assembly start (001 to 499 by default), and **New Assembly** starts at 500, so assembly numbers are easy to recognise.

Every file in the cad folder is named after its part and revision: HCR-100010-001_A.SLDPRT, HCR-100010-001_A.SLDDRW, HCR-100010-001_A.STEP, and HCR-100010-001_A.PDF. Litterbox uses the names to match files to parts and revisions, and ignores files that don't follow the pattern.

Create Variant (Create menu) copies the highlighted part to a new variant of its part number, as a part of its own: a bracket in two lengths ordered as two items, for example. The copy starts at Rev A with the original's name (or one you type), its SolidWorks files are copies of the latest revision's (the drawing points at the copied model), and it is checked out to you.

Projects

A **project** reserves a block of part numbers: with 6 digits, project Rover starting 12 owns 120000 to 129999. Choose a project in New Part, New Assembly, or Import, and Litterbox suggests the next number in its block. With no project, Litterbox never hands out a number inside a block.

Admins and the Owner add projects (New Project...), or anyone on the team when an Admin allows it in Team > Vault Settings... (**Who may add projects**). Whoever adds a project is its Lead to start with. Vault Settings can also set **a project's starting digits**: the fewest a new project may have, so nobody sets aside too many numbers (with 6 digits, a project starting 1 owns 100000 part numbers). Anyone else is refused fewer; an Admin is warned and may Proceed. Projects already made keep their digits. Either setting makes the vault need Litterbox 0.6.4.

Team > **Projects...** lists the projects you can see. Each project can have a **Project Lead**, **members**, and two options. All of these are optional:

Option	Effect
Restrict Project	Only its members, its Lead, Admins, and the Owner can make part numbers in it, change its parts (check out, check in, release, revise, void, and the other actions), and ask for or sign its workflows. Superusers can still sign. A restricted project appears in New Part, New Assembly, and Import only for those who can use it.
Hide Project	Restricted, and invisible to everyone else. Litterbox leaves its parts (records, CAD files, and cancelled revisions) out of their copy of the vault, so their CAD files are never downloaded, and leaves its ECOs out of their ECOs screen.
Only the Lead changes its folders	Members see the project's folders in Home but can't change them; its Lead, Admins, and the Owner can.

Either option needs a Lead. Admins and the Owner add, change, and delete projects. A project's Lead can change everything about it except its digits, including its members. In the Project window, search the team, select people, and press **Add**; select members and press **Remove** to take them off.

A part belongs to the project whose block holds its number. Someone added to a hidden project gets its parts at their next Get Latest, and someone taken off loses them at theirs.

> Restrict and Hide control what Litterbox does. They are not security: anyone with access to the vault on the Git host can read all of it, including a hidden project's records.

Finding parts

- The sidebar views filter the list: your checkouts, everything checked out, released parts, parts not downloaded, and void parts (which no other view shows).
- The search box looks in every column, or in one column chosen beside it. `*` matches any characters and `?` matches one, against the whole value (Te*rt finds Test Part but not My Test Part).
- Click a column heading to sort. Part number, Rev, Name, Description, Files, and Kind show to start with. View > Columns adds Project, COTS, Status, Checked out by, Released rev, Imported from, Material, and the purchasing info: Supplier, Supplier part number, Unit cost, and Lead time (days). Unit cost and Lead time sort as numbers.

Folders

Home (the Home / Vault switch above the list) shows folders instead of the parts list. A folder lists parts, documents, and ECOs, one line each: a tick box, the number, and the name. A folder only points at them, so the same part can be in any number of folders, and removing it from a folder, or deleting the folder, never touches the part.

Top	What it holds
Personal	Your own folders. They are kept on your computer only (C:\Litterbox\folders), so nobody else sees them, and a new computer starts with none.
Team	The org's folders. Everyone sees them. Admins, the Owner, and people with a role ticked **Team folders** (Team > Roles...) change them.
Projects	A folder for each project you are the Lead or a member of (Admins and the Owner see every project that isn't hidden). Its Lead changes it, and its members too, unless the project is set so only its Lead does.

- Right-click a top or a folder for **New Folder...**, **Rename...** and **Delete Folder...** (folders only), **Add Parts...**, and **Put the ticked here**. Right-click an item for **Details...**, **Remove from This Folder**, and the part actions.
- **Add Parts...** opens a search window of every part and document not in that folder yet. Type to search by number, name, or description, select one or several (Ctrl or Shift), and press **Add selected**.
- Parts, documents, and ECOs can go straight in a top (Personal, Team, or a project), outside every folder: right-click the top, drag onto it, or choose it in Add to Folder.
- In a top or a folder, the folders come first, then parts, documents, and ECOs, each by number.
- **Add to Folder...** in the Part menu, on the Documents screen, and on the ECOs screen puts the ticked parts (or the highlighted one) in a folder (or a top) you choose, or in a new folder.
- Drag an item or a folder onto a folder to move it. Dragging an item into another top (from Personal to Team, say) copies it.

- Ticks are shared with the parts list, so Check Out, Release, and the other actions work on what is ticked in Home. Clicking a part shows its details; double-clicking opens them.
- Search works in Home too: it keeps what matches and the folders it is in.

Team and project folders are uploaded like any change, and others see them at their next Get Latest. Two people can change folders at the same time without a clash.

Checking out and checking in

Files you don't have checked out are **read-only** on your disk, so SolidWorks opens them read-only and you can't save over someone else's work. To edit a part:

1. Tick it and press **Check Out**. Litterbox takes the lock on the part's record and files, downloads the latest version if needed, and makes the files writable. A part already open in SolidWorks is made writable there too.
2. Edit and save in SolidWorks.
3. Press **Check In** to upload your changes and release the lock, or **Save Progress** to upload them and keep the lock. A window lists the parts about to go up (number, revision, and name) and asks what you changed; the comment goes in each part's history, and may be left blank.

Undo Checkout discards your changes and restores the part as it is in the vault. Check In asks before checking in a part with no changes. It refuses a part saved in SolidWorks after Check In was pressed, because the save could be partly uploaded; that part stays checked out, and you check it in again.

> Don't save parts in SolidWorks while they are being checked in. Wait until the status bar says Ready.

When an assembly is checked in with SolidWorks open, Litterbox reads which parts it uses and how many of each, and which Toolbox parts and files from outside the vault it uses. When a part is checked in with SolidWorks open, Litterbox reads its material. It also reads the model's configurations (see Configurations). Without SolidWorks the check-in still works, and Litterbox marks these as needing an update (see Assemblies, references, and BOMs, and Materials).

If the team has a **Toolbox rule** (see Toolbox parts), Check In refuses an assembly that breaks it, says which files are the problem, and leaves it checked out to you.

Getting other people's changes

Get Latest (Vault menu, or the change count in the header) downloads what others have uploaded. It updates the files you already have and doesn't fetch the rest. If it replaces a file that is open in SolidWorks, close and reopen the file there. **Refresh** (F5) re-reads your copy without contacting the host.

Opening files

The **Part** menu has Open, Open Drawing, and **Open PDF**. When a part has a working revision and released ones, Litterbox asks which to open. Files open in the program Windows uses for them. To use another program for a file type, add it to settings.yaml:

```
open_with:
  .pdf: C:\Program Files\Adobe\Acrobat\Acrobat.exe
```

The details pane

Tab	Contents
Details	For the chosen revision: its state, who released it (with a workflow, whoever asked for the release: every workflow is recorded as done by whoever asked) and when, the workflow and signers, its ECO, what its release changed (Form, Fit, Function), the disposition of its parts after a later release, its files, material, and STEP/PDF, and the part's purchasing info.
History	Every revision, with buttons to open older ones, and every event in the part's history.
Uses / Where used	The parts an assembly uses, and the assemblies that use this part, with whether each use is up to date.
BOM	The bill of materials of an assembly revision (see below).
Documents	Documents linked to the revision chosen on the Details tab.

Revisions and the lifecycle

A part is **in work** until it is released. Its first revision is A. The Lifecycle menu has these actions:

Action	Effect
Release	Freezes the working revision. Its files become read-only for everyone, and it can't be checked out. Litterbox first checks that the files exist, and warns (or, if the org chooses, refuses) when an assembly uses parts that are unreleased or out of date.
Revise	Starts the next revision of a released part: copies its files to the new letter (B), points drawings at the new model, and checks it out to you. The released revision is unchanged.
Cancel Revision	Discards a revision that was never released. Its files move to the cad\cancelled-revs folder (read-only, kept for the history), and its letter isn't reused.
Unrelease	Returns a part to in work, with a reason, when a release was a mistake.
Void	Retires a part that was never released. Its number is never reused.
Make COTS	Records a part as commercial off-the-shelf (bought, not modified). It is frozen like a release.
Edit Details	Changes a part's name and description, and switches it between CAD and non-CAD until its first release.

Revision letters skip I, O, Q, S, X, and Z, which are easily mistaken for numbers (as ASME Y14.35 recommends): H is followed by J, and Y by AA.

Release, Unrelease, Void, and Make COTS work on several ticked parts at once, and each part's history records who did it. If the vault has a workflow for the action, pressing it starts a sign-off request instead (see Sign-off workflows and ECOs).

Form, Fit, Function, and Dispo

When a workflow with an ECO releases a part at Rev B or later, the Workflow window asks two things for each part:

- **Form, Fit, and Function**: which of these the new revision changes. Leave all three clear for a change that affects none of them, such as a drawing correction.
- **Dispo** (disposition): what to do with parts already made to the previous revision: **Use As Is**, **Rework**, **Scrap**, or **Return To Supplier**. It starts blank and must be chosen.

A void through such a workflow asks for a Dispo of the voided revision's parts. A first release (Rev A) asks neither. Workflows without an ECO, and Release or Void without a workflow, don't ask them.

What changed is stored with the new revision, and the Dispo with the old one. Choose the old revision on the Details tab to see it, for example "Disposition: Scrap (Rev B released)".

Assemblies, references, and BOMs

Uses and where used

Litterbox keeps a list of the parts each assembly revision uses, read from the assembly at check-in. This list provides **Uses** and **Where used** on the details pane, and Part > **References...**. Each use shows whether it is up to date: a newer revision may be released, or the part used may be unreleased or void.

Update references... points in-work assemblies at the latest released revisions of their parts, in SolidWorks, and checks them in. It never changes a released assembly; revise it first.

The BOM tab

An assembly's bill of materials is worked out from those lists each time it is shown, so it always matches the models. It has three views:

- **Single level**: what the assembly itself uses.
- **Indented**: the same, with each sub-assembly's parts under it.
- **Every part once**: each part revision in the tree once, with its total quantity.

Each line shows the revision, quantity, name, material, supplier, unit cost, total cost, lead time, and whether the use is out of date. The line above the table gives the assembly's **total cost**, from every part in the tree once, and says how many parts have no unit cost or no count. **Export...** saves the current view as a .csv file, to a folder outside the vault.

Quantities are counted by SolidWorks at check-in: each copy at the top level of the assembly, excluding suppressed copies and those excluded from the BOM. If an assembly was checked in without SolidWorks, the BOM says it needs updating, and **Update BOM** counts it again (starting SolidWorks in the background if needed).

When an assembly or a kit is released, Litterbox saves its indented BOM in the cad folder as HCR-100010-500_A_BOM.csv, as a fixed record of the released revision: what it was made of, with names, quantities, and costs as they stood at release. If it can't be saved then (the upload failed, say), the part is flagged **BOM file needs saving**, and Part > **Save Released BOM** (or **Save BOM File** on the Details tab) saves it later, with names and costs as they are when it is saved. A BOM saved with its release is never saved again over it.

Toolbox parts

An Admin chooses the team's **Toolbox rule** in Team > Vault Settings...:

Rule	An assembly may use
No Toolbox parts	Only parts in the vault: no SolidWorks Toolbox parts, and no files from outside the vault. A new vault starts with this rule.
Only the team's Toolbox	Toolbox parts from the team's Toolbox in the vault (see The team's Toolbox), and no other files from outside the vault.
Don't check	Anything. Vaults made before Litterbox 0.6.0 start with this.

Check In refuses an assembly that breaks the rule, and so does **Release**, since a rule can be set after an assembly was checked in. An assembly checked in without SolidWorks open hasn't been looked at, so Release asks for **Update BOM** first (on the BOM tab), which reads it with SolidWorks.

SolidWorks takes every Toolbox part from the Toolbox folder it is set to use (Tools > Options > System Options > Hole Wizard/Toolbox), whatever folder the assembly was saved with. So with Only the team's Toolbox, set SolidWorks to the team's folder once on each computer. Litterbox says so after the first refresh if it isn't.

Non-CAD parts and kits

New Part and New Assembly have a **Kind**: CAD or **Non-CAD**. A non-CAD part (paint, glue, wire, raw stock, labels) has a part number, revisions, a lifecycle, and purchasing info, but no files.

On a checked-out CAD assembly, the BOM tab's **Add item...** adds a non-CAD part with a quantity, and **Remove** takes it off. A **kit** is a non-CAD assembly made only of lines added this way, from any parts. Added lines freeze at release and carry on to the next revision. A released kit's BOM is saved with it like an assembly's: its only file.

Purchasing, costs, and materials

Purchasing info

Lifecycle > **Purchasing Info...** sets a part's supplier, supplier part number, unit cost, and lead time in days. Purchasing info belongs to the part, not to a revision, so it needs no check-out and works on released parts. The change is uploaded at once and recorded in the part's history. View > Columns can show it in the parts list. An assembly's columns show its own purchasing info, not a total of its parts' (the BOM tab adds those up).

An Admin chooses the **team's currency** in Team > Vault Settings.... Costs are shown with its symbol ($12.50) on the Details tab, in the BOM, and beside the Unit cost box. Unit costs are stored as plain numbers, so changing the currency doesn't convert them. An exported BOM names the currency in its column headings and keeps the cells as numbers.

Materials

Each revision's material is read from SolidWorks (its active configuration) when the part is checked in with SolidWorks open. It appears on the Details tab, in the BOM, and in the Material column. A part with no material assigned shows blank. A part checked in without SolidWorks, or before Litterbox read materials, shows **Not read yet**; tick it and choose Part > **Update Material** to read it.

Release outputs and exports

STEP and PDF files

- **At release.** After a part is released, Litterbox makes a STEP (AP214) of the model and a PDF of every sheet of the drawing from the frozen files, and uploads them as HCR-100010-001_A.STEP and _A.PDF. If the person releasing has no SolidWorks, the release still happens, the part is flagged **STEP/PDF needs updating**, and anyone with SolidWorks can make them later.
- **On demand.** Part > **Generate STEP/PDF...** makes them for the ticked parts, into the vault or into a folder outside it. An Admin can tick Make released parts' files again to replace damaged ones.

Litterbox starts SolidWorks in the background if it isn't running, closes it afterwards, and restores your STEP settings.

Exporting

Part > **Export Files...** copies the ticked parts' files to a folder you choose: any of model, drawing, STEP, PDF, and the BOM saved at release, from the latest released revision, the working revision, or a revision you pick, and optionally the parts each assembly uses. Exports always go outside C:\Litterbox and outside any Git repository.

Documents

The **Documents** screen keeps documents such as test procedures, plans, datasheets, and review slides under revision control. Documents don't use part numbers. An Admin sets up **document types** in Team > Document Types... (a name and a prefix, such as Propulsion Document, PROP), and **New Document** numbers each document with the next free number for its type: PROP-0001, PROP-0002, and so on.

Documents go through the same lifecycle as parts: check out, check in, release, revise (which copies the files to the new revision), unrelease, void, and cancel revision. **Add Files...** adds a file of any type to the working revision (one per type, such as PROP-0001_A.docx and PROP-0001_A.pdf), **Remove File...** removes one, and **Open** opens one. **Details...** shows the document's revisions, files, history, and the part revisions that link to it.

Links to parts. A part's Documents tab lists the documents linked to a revision, with Open, Add..., and Remove. A link belongs to the part revision: it freezes at release and carries on to the next revision. On a checked-out part, links can be added and removed freely. An Admin or a superuser can add a link to a released part at any time; removing one from a released part needs a Remove document link workflow.

An Admin can hide a document type (Hide / Offer in Document Types). Its documents are unchanged, and New Document no longer offers it.

Sign-off workflows and ECOs

How workflows work

A workflow is a list of sign-offs that an action needs before it happens. Each sign-off asks for a person with a role (Responsible Engineer) or with any of several roles (Manager or Chief). An action without a workflow happens when you press it. An Admin sets up workflows in Team > **Part Workflows...** (Release, Unrelease, Void, and Make COTS) and **Doc Workflows...** (Release document, Unrelease document, Void document, and Remove document link). An action can have several named workflows. **Add standard workflows** adds a starting set.

Each workflow also sets whether one person may fill more than one sign-off, whether the requester may sign, and whether it **requires an ECO**.

Asking for sign-off

1. Tick the parts and press the action (Release, for example). With a workflow in place, the Workflow window opens.
2. Check the parts list: Part, Name, and Rev ("A → B" for a new revision). If the request has an ECO, the list also has **Form**, **Fit**, **Function**, and **Dispo** for parts at Rev B or later; click a cell to tick it or choose from its menu. Add Parts and Remove Parts change the list.
3. Choose the workflow, a person for each sign-off, and any extra sign-offs. If the request needs an ECO, enter its title and description.
4. Submit. Several parts go through one request, and each person signs once for all of them. The parts show as waiting for sign-offs, and everyone named sees the request in **Workflows > My Tasks**.
5. When the last person signs, the action happens. If it fails (for example, offline), the sign-offs are kept and **Complete** in My Tasks finishes it.

My Tasks and All Workflows show new requests (and changes to them) at every refresh, before Get Latest. One your copy hasn't got yet says so under the list; Review, or any of its buttons, gets the latest first.

Only people who may sign for the parts' projects are offered as signers (see Projects).

The requester or an Admin can withdraw a request until it is done. An Admin can **Override Approve** or **Override Reject**, recorded as an override with a reason, and can change who signs in **All Workflows** (Change signer...).

Reviewing a request

Select a request in **My Tasks** and press **Review**, or double-click it. The Review window shows the request as it was submitted, read-only: the parts (with Form, Fit, Function, and Dispo when it has an ECO), the workflow, every sign-off with its person and whether it is signed, and the ECO.

- **Open Drawing** and **Open PDF** open the selected part's files at the revision being released. **Previous Drawing** and **Previous PDF** open the revision before it.
- Each of your sign-offs has a **Reassign** button to give it to someone else with the role. **Undo** takes it back. Nothing is saved until you press a button at the bottom.
- The **Comment** is optional to approve and required to reject. The requester sees it.

Button	Effect
Approve	Hands any reassigned sign-offs to their new people, then approves the rest of yours. Unavailable once all of yours are reassigned.
Reject	Sends the request back to the requester and drops any reassignments. Needs a comment. Unavailable once all of yours are reassigned.
Reassign	Available once all of your sign-offs are reassigned. Hands them over without signing.

A person you hand a sign-off to sees the request in their My Tasks.

Rejected requests

A rejected request leaves the signers' My Tasks and appears in the requester's, marked rejected, with who rejected it and why. It is then a **draft**: its parts can be checked out and in, but can't go into another workflow.

Review opens the draft in the Workflow window as it was submitted. Change the parts, people, Form, Fit, Function, Dispo, or ECO, and press **Resubmit workflow**; everyone signs again, and the same ECO continues. **Withdraw** drops the request.

Superusers. An Admin can make a role a superuser role (Team > Roles...). A person with that role can give any sign-off on any request they can see, recorded as "as Quality for Bea Jones (superuser)". Admins, the Owner, and superusers see **All Workflows**.

Engineering change orders

A workflow that requires an ECO asks for a **title** and a **description** when the request starts. Litterbox saves the ECO as ECO-0001, ECO-0002, and so on, with each part's revision, what changed, and its Dispo. A release whose parts are all at Rev A needs no ECO, even if its workflow requires one.

An ECO stays open while its request waits, and while a rejected request is being changed (its history records the rejection and the resubmission). It closes, with the sign-offs and comments, when the request is released or withdrawn, and is not changed after that. The **ECOs** screen lists and searches them, and a released revision's Details tab links to its ECO.

Running the team

This chapter is for Admins and the Owner. These tasks are on the **Team** screen. Changes to people, roles, and workflows are drafted there and uploaded together with Save.

People and access

Access	Permissions
Owner	Everything, plus renaming the org and transferring ownership. There is exactly one Owner.
Admin	Manage people, roles, workflows, projects, document types, the team's Toolbox, templates, and vault settings; override workflows; break a checkout; remake released STEP/PDF files; link documents to released parts. Admins can work in every project, restricted or hidden.
Member	Everyday work: create, check out, check in, release (subject to workflows), and sign off. A project's Lead can change that project and its templates.
Not listed, or switched off	Read-only: look, open, and export.

Team > **Roles...** lists the roles the org uses. A role ticked **Superuser** may sign any sign-off; a role ticked **Team folders** may change the Team folders in Home.

Litterbox identifies you by the account you are signed in to on the Git host, matched against each person's host user names. **Who am I?** checks it. An Admin can require **signed commits** (Vault Settings); each person then sets up a signing key with Signing..., and Litterbox refuses uploads that aren't signed by a key on the team list.

When someone can't check a part in, an Admin can **Break Checkout** (Lifecycle menu). That person's changes aren't uploaded. At their next sync, Litterbox saves their copies of files that changed beside the vault and tells them where.

Vault settings

Setting	Controls
Organization name	The name shown beside the org code. Only the Owner can change it.
Commits for a batch	One commit per part or one for the whole batch.
Release rule	Warn, block, or do nothing when a release uses parts that are out of date or unreleased.
Signed commits	Whether uploads must be signed by a key on the team list.
Assembly variant start	The first variant New Assembly uses (500).
SolidWorks version	The version everyone works in. People with a newer SolidWorks can't check out or revise, because SolidWorks upgrades files when it saves them.
Team's currency	The symbol costs are shown with.
SolidWorks Toolbox parts	The Toolbox rule (see Toolbox parts). Setting one makes everyone need Litterbox 0.6.0 or newer.

The team's Toolbox

With the rule Only the team's Toolbox, the team shares one copy of the SolidWorks Toolbox, kept in the vault's **toolbox** folder (C:\Litterbox\<vault>\toolbox). Team > **Toolbox...** shows it: how many files it has, how many are downloaded on your computer, and whether your SolidWorks uses it. Everyone can open it and press **Download**. Only Admins and the Owner can change it:

1. **Copy in a Toolbox folder...** makes the team's Toolbox a copy of a Toolbox folder (C:\SOLIDWORKS Data, say) and uploads it. A whole Toolbox is several hundred MB, so this takes a while. Copying in again replaces it.
2. Everyone sets SolidWorks to use the team's folder (Tools > Options > System Options > Hole Wizard/Toolbox).
3. To add sizes or change Toolbox's settings, press **Let me change it**, make the changes in SolidWorks, then press **Upload changes**. Set Toolbox to make a file for each size (Toolbox Settings > User Settings > Create Parts), so a new size is a new file rather than a change to a shared one.

The team's Toolbox is read-only on everyone's computer, and Litterbox refuses to upload a change to it from anyone but an Admin or the Owner. An assembly can't be checked in with a Toolbox part that is on your computer but not uploaded to the team's Toolbox yet.

Vault maintenance

- **Vault > Update file rules...** appears when Litterbox has newer rules for which files Git LFS stores and locks. An Admin is asked once per session. The update changes only Litterbox's own lines. Version 4 (Litterbox 0.6.3) also moves the vault's cancelled revisions from the old cancelled-revs folder at the top of the vault into cad\cancelled-revs, with their history, and makes everyone need 0.6.3.
- **Vault > Upgrade records...** appears when an update of Litterbox keeps part records in a new format. Until an Admin upgrades the vault, nobody on the new version can change anything, so an Admin is asked once per session. Upgrading makes everyone update Litterbox, and rewrites every record in one upload. Parts checked out at the time are upgraded when they are checked in.
- **Vault > Vault Size...** shows how much the vault uses on the host (its history and every version of every Git LFS file) and how much is downloaded on your computer.
- Some features raise the oldest Litterbox version the vault accepts (workflows, projects, documents, the Toolbox rule, and others). Litterbox tells anyone on an older version to update before they upload anything.

Importing existing CAD

Create > **Import...** brings a folder of SolidWorks files into the vault. It gives each part a number and renames the files to match, updating their references so assemblies still find their parts.

1. Choose the folders to import, or a saved scan. Litterbox scans them with SolidWorks to find the parts, assemblies, and drawings, what each assembly uses, and which files the vault already has.
2. Review the table. Each item has a proposed part number, which you can change, and a name. Choose a project (only projects you can use are offered), or import a family under one base number. Untick anything you don't want.
3. **Finish & Import** copies the files into the vault under their new names through SolidWorks, creates the records, and uploads everything. STEP and PDF files with the same name as a model or drawing are included.

Import COTS... does the same for purchased parts and records them as COTS. If an import stops part-way, Litterbox records what it did and finishes or cleans up next time.

> Try an import on a copy of a small, self-contained assembly first. Imported files are renamed, so files outside the import that point at the old names won't follow them.

Working with SolidWorks

Litterbox uses the SolidWorks API to read references, count parts, find Toolbox parts, read materials, copy files at Revise and Import, make new files from templates, and make STEP and PDF files.

- If SolidWorks isn't running when Litterbox needs it, Litterbox starts it **in the background** and closes it afterwards. If it is running, Litterbox uses it and leaves your open documents as they are.
- Check-in only uses a SolidWorks that is already running.
- Litterbox reads your installed SolidWorks version and its Toolbox folder from Windows, without starting it, for the org's version check and the Toolbox rule.
- Litterbox can **watch** what SolidWorks has open (`solidworks_watch: true` in settings.yaml). It then warns before checking in parts with unsaved changes, and before Get Latest replaces an open file.

Configurations

A part or assembly file can have several **configurations**. Litterbox checks out, revises, and releases files, so a file's configurations always move together: changing one makes a new revision of the whole file. A part's Details tab says when it has several. When a configuration needs its own part number, use **Create Variant** instead (see Part numbers and file names).

- **Named after the part.** With SolidWorks open, Check In renames a file's configuration to the part's Name when it is the file's only configuration and is called Default, so an assembly's tree shows HCR-100010-001_A<1> (Bracket). When the Name changes, it is renamed again at the next check-in. Import does the same to the files it copies in. A file with configurations of its own is left alone, and so is a file open in SolidWorks with unsaved changes. Assemblies follow a renamed configuration. The configuration's BOM part number is set to the document name, so a SolidWorks BOM still shows the part number.
- **Warnings at check-in.** Litterbox says when a configuration that an assembly uses was deleted from a part (SolidWorks would show another configuration there), when a part's configurations have different materials (Litterbox keeps one material per revision: the configuration's the file was saved in), and when a part is linked to a design table or equations file outside it (Litterbox can't keep that file with the revisions; embed it instead). Update references says when the newer revision has lost a configuration an assembly uses.

Templates

Litterbox makes a part's files from the team's SolidWorks **templates**, so every part, assembly, and drawing starts with the same units, materials, drafting standard, and title block. There are three kinds: part templates (.prtdot), assembly templates (.asmdot), and drawing templates (.drwdot).

- **The org's and each project's.** The org has its own templates, and each project can have more. Among the org's templates of one kind, one is the **default**, and the same among each project's. A part in a project gets its project's default if the project has templates of that kind, otherwise the org's default. Every other template of the project and the org can be chosen instead.
- **New Part and New Assembly** have a **Template** box, with the default chosen. Litterbox makes the model from it and opens it. Choose None to save the model yourself.

- **Create > New Model...** makes the model of a part checked out to you that has none yet (one created with None, say), and **Create > New Drawing...** makes the drawing of a part that has a model and no drawing. Both ask which template, with the default chosen. A new drawing is an **empty sheet** of the template's size, with its title block: add the views in SolidWorks.
- The file is made in the cad folder under the part's name for its working revision, like a file you saved there, and goes up with your next Check In. Litterbox needs SolidWorks on your computer to make it (it starts SolidWorks in the background if it isn't running).

Team > **Templates...** shows the org's templates and each project's. Everyone can look. Admins and the Owner change the org's; a project's Lead (and Admins, and the Owner) changes the project's:

- **Add...** copies a template file into the vault's **templates** folder and uploads it: the org's at the top of the folder, a project's in a folder of the project's name (templates\Rover). The first template of a kind becomes the default. The org and each project can't have two templates of the same file name, but the org and a project can.
- **Make Default** makes the selected template the default of its kind.
- **Remove...** takes a template out of the vault. Files already made from it don't change. If it was the default, the next one by name becomes the default.

Litterbox refuses to upload a template change from anyone else. A renamed project keeps its templates: its folder is renamed with it.

Things to watch

- **Keep the sheet format in the drawing template.** SolidWorks saves the sheet format (the border and title block) inside a drawing template, so a drawing made from it has the title block even on a computer without the sheet format's .slddrt file. The template also remembers where that .slddrt file was, and SolidWorks only looks there when someone reloads or changes the sheet format. Keep the team's .slddrt files where everyone has them at the same path.
- **Save templates in the org's SolidWorks version, or older.** A template saved in a newer SolidWorks can't be used by an older one; Litterbox says so when it can't make a file. When the org moves to a newer SolidWorks, the templates can stay as they are.
- **Title blocks.** Litterbox names each file after its part and revision (HCR-100010-001_A), and, with SolidWorks open at check-in, names a file's only configuration after the part's Name (see Configurations). A title block that shows the file name and the configuration name ($PRP:"SW-File Name" and $PRP:"SW-Configuration Name", or $PRPSHEET for the model on the sheet) therefore shows the part number, revision, and Name without anyone typing them. Litterbox doesn't write other properties into the files.
- **Units, materials, and drafting standards come from the template.** Set them in the template, not part by part, so new parts agree.
- **A changed template only changes new files.** Existing drawings keep their sheet format; to bring one up to date, reload its sheet format in SolidWorks while it is checked out.
- **SolidWorks' own File > New** still uses the templates SolidWorks is set to (Tools > Options > System Options > Default Templates and File Locations > Document Templates). Point Document Templates at the vault's templates folder (C:\Litterbox\<vault>\templates), and your project's folder inside it, so File > New offers the team's templates too.

How Litterbox works

This chapter is for the curious, and for whoever looks after the vault. Nothing in it needs doing by hand.

The vault is a Git repository

A vault is a Git repository on the team's Git host, with a copy on each computer in C:\Litterbox\<vault>. Everything Litterbox keeps is a file in it:

Folder	What it holds
PN-logs, DOC-logs	One record per part or document (HCR-100010-001.yaml): its name, revisions, history, purchasing info, and what it uses. Plain text.
cad, documents	The parts' and documents' files, named after them (HCR-100010-001_A.SLDPRT).
ECO	Engineering change orders.
cad\cancelled-revs	Revisions cancelled before release, kept for the history (at the top of the vault before 0.6.3).
folders	The Team and project folders: one small file per change, never edited, so two people never clash.
templates	The org's and projects' SolidWorks templates, and templates.yaml saying which is the default.
toolbox	The team's SolidWorks Toolbox, if it keeps one.
.litterbox	The shared settings (config.yaml), the team (people.yaml), workflows, and document types.

Large files (CAD files, PDFs, pictures, and templates) are stored with **Git LFS**, Git's add-on for large files: Git keeps a small pointer, and the file itself is kept once on the host. The vault's .gitattributes file says which files go to Git LFS and which can be locked; Vault > Update file rules... keeps it current.

Locks, read-only files, and uploads

- **Check Out** takes Git LFS **locks** on the part's record and files. The host keeps the locks, so only one person can hold a part at a time, and Litterbox makes those files writable for them alone. Every other file of a part is **read-only** on your disk; Litterbox puts that back after every download, check-in, and undo.
- Every change is a Git **commit** with your name and a message ("HCR-100010-001: Thicker flange"), so the history says who did what, when, and why. If the team requires signed commits, each is signed with your key.
- An **upload** first gets everyone else's changes (a sync), then checks the upload is allowed: you are still on the team, your Litterbox is new enough for the vault, and any changes to the Toolbox, folders, or templates are ones you may make. Then it pushes. If someone else uploaded in the meantime, Litterbox syncs and tries again. If it still can't, your commit stays on your computer and goes up with the next upload; nothing is left half done.

- Actions on several parts (Check In, Release, and others) do one sync, one commit, and one upload for the whole batch, while each part still gets its own history entry. A part that can't be done is reported, and the rest carry on.
- Check In remembers each file's size and time when you press it, and refuses to upload a part saved in SolidWorks after that, so a half-saved file is never uploaded.
- Long operations (Import, Revise) write a small note in the vault's .git folder before they start. If Litterbox stops part-way, the note says what to finish or undo next time.

Downloads

Litterbox gets other people's changes at Get Latest and before every upload. With **download on demand** (the usual setting), Git leaves every CAD file as a small pointer until it is needed, and Litterbox downloads exactly the files it needs: those you open or check out, and what an assembly uses. Files you already have stay downloaded at each Get Latest. A hidden project's files are left out of the copies of people who aren't in it (Git's sparse checkout), so they are never downloaded.

The job queue

Everything that touches the vault or SolidWorks runs in the **background**, so the window never freezes: you click, the work becomes a **job** in a queue, and you can carry on browsing, ticking, and queueing more.

- Jobs run **one at a time, in the order they were queued**, because Git can change only one thing in a vault at once. The status bar shows the job running, and how many more are waiting ("Checking in 3 parts - 2 more waiting"); a job queued behind others shows "Queued: ...".
- Each job knows the parts it changes. Those parts are **busy** until it has finished and its result is shown: their status says so, and other changes to them are refused until then. Looking at a busy part (Open, Details, Export) still works.
- A **refresh** (re-reading your copy of the vault) is queued like any job. A newer refresh replaces one still waiting, so they never pile up.
- When a job finishes, the window shows the result and refreshes the list. If it **failed**, there is no pop-up (you may be busy elsewhere): the status bar's dot turns red and stays red until you press **Details...** and dismiss it, so a problem isn't missed during a break. Jobs queued after it still run.
- A few things need the vault to themselves, so they can't be used while jobs are queued or running: switching org, Import, and Get Latest from the header.
- SolidWorks work in a job uses a SolidWorks that is already running, or starts one in the background and closes it when the job is done.
- With `debug_mode: true` in settings.yaml, the status bar shows how long a job has been running, so a slow host looks slow rather than stuck.

Settings and troubleshooting

Your settings

Your personal settings are in C:\Litterbox\settings.yaml, outside every vault, and are not shared. Most are set in the program (columns, text size, and the open org). These are only in the file:

Setting	Effect
update_check: false	Don't check for a newer Litterbox at start-up.
update_channel: beta	Offer betas too (set by Help > Get beta versions).
download: all	Download every CAD file, not only those you use.
open_with	The program that opens each file type.
solidworks_watch: true	Watch what SolidWorks has open (see Working with SolidWorks).
solidworks_popups: false	Send the watcher's warnings to the log instead of pop-ups.
debug_mode: true	More detail in the log, including how long each step took.

View > **Text size** (Ctrl+= and Ctrl+-) changes the size of everything. Litterbox follows Windows' display scaling on each monitor.

Problems

- A problem turns the status bar's dot red. **Details...** says what went wrong and what to do.
- **Help > Report a problem...** (or **Report...** in the Details window) lists exactly what goes in the report (each file, about how big, and what it holds), asks what you were doing, then makes the report and opens an email to the Litterbox team with it, for you to send. Nothing is sent until you press Send. With Outlook, the report is attached and Litterbox offers to open it first; otherwise attach it yourself (Explorer shows it).
- The report holds report.txt (your comment, the versions of Litterbox, Windows, Python, SolidWorks, Git, and Git LFS, the problems in the status bar, and this session's unexpected errors), Litterbox's log, your Litterbox settings, the vault's shared settings, and CONTENTS.txt listing them. Untick **Include Litterbox's log** or **Include your Litterbox settings...** to leave those out. The log names parts, files, and check-in comments, and can name people on your team. Passwords and tokens are taken out of every file, and no CAD files, documents, part records, or the team list ever go in. Reports are kept in C:\Litterbox\logs\reports.
- The **Log** (View > Log, or the status bar) shows every Git command and its result. The full log is C:\Litterbox\logs\litterbox.log; include it when you ask for help.
- If an upload is refused because someone else uploaded first, Litterbox gets their changes and tries again. If that fails, your commits are kept and nothing is left half done; try again later.
- If Git LFS's lock cache is damaged, Litterbox moves it aside and continues.
- An action on a part in a restricted project you are not a member of is refused, with the name of the project's Lead to ask.

The command line

The same features run from a terminal in the app folder of a source copy: `python -m litterbox <command>`, with status, the part actions (checkout, checkin, release, and others), new-part, scan, import, and init-vault. `python -m litterbox <command> --help` lists each command's options.

Quick reference

Keys	Action
Ctrl+1 ... Ctrl+5	Parts, Workflows, ECOs, Team, and Documents
F5	Refresh
Ctrl+= / Ctrl+- / Ctrl+0	Larger, smaller, and default text size
Alt+F, Alt+A, Alt+V, Alt+H	Open the File, Vault, View, and Help menus
Double-click a part	Open its details in a separate window
Double-click a request in My Tasks	Review it
Shift-click a tick box	Tick every part between the last click and this one
Space in Home	Tick or untick the highlighted item